CMPE492 - SENIOR PROJECT II
Final Report, Spring2026


**Project Team Members:**

Levent BEKTEMUR, Database Manager
Dilge Nisa ÜSTÜNTAŞ, Project Manager
Süzanna NEBİOĞLU, Product Designer

**Supervisor:**

Ulaş Güleç

**Jury Members:**

Tolga Kurtuluş Çapın
Tansel Dökeroğlu

**Coordinator:**

Dr. Özlem ALBAYRAK


A
15

# Table of Contents

**1- INTRODUCTION and SCOPE 3**
**2- ARCHITECTURE and DESIGN 3**
Current Design 3
1. Frontend Web Application Layer 3
2. Backend API Layer 4
3. Unity WebGL Visualization Engine 4
System Workflow 4
Deviation from Low Level Design 5
Enhanced Authentication System 5
Database Integration 5
Modern Technology Stack 5
Enhanced File Management 5
**3- TEST RESULTS 6**
Functional Tests 6
Interface Tests 7
Security Tests 7
Figure 3: Security tests' results table. 8
Validation Tests 8
Test Assessment 9
Summary Statistics 9
Known Limitations 9
Potential Future Enhancements 10
**4- TOOLS and TECHNOLOGIES USED 10**
Programming Languages 10
Frameworks and Libraries 10
Development Tools 11
Infrastructure 11
Engineering Standards and Practices 11
**5- ENGINEERING IMPACT and CONTEMPORARY ISSUES 12**
Societal Impact 12
Economic Impact 12
Environmental Impact 12
Global Impact 12
Contemporary Issues 12
Web-Based VR Evolution 12
Security in Web Applications 13
Performance Optimization 13
Data Privacy and Compliance 13
**6- LESSONS LEARNED 13**

Technical Skills 13
Engineering Practices 13
Project Management 14
Key Insights 14
**6- REFERENCES** **15**

RELATED DOCUMENTS

| **Name** | **URL** |
|---|---|
| Low Level Design | https://humanfromnowhere.github.io/discover.github.io/ |
| High Level Design | https://humanfromnowhere.github.io/discover.github.io/ |

CHANGE HISTORY

| Date | Changed by | Change Description |
|---|---|---|
| 10.05.2026 | Süzanna Nebioğlu | Created Initial Draft. |
| 10.05.2026 | Dilge Nisa Üstüntaş | Wrote Introduction and scope part. Added the test results. |
| 10.05.2026 | Levent Bektemur | Wrote Tools and Technologies Used part. |
| 11.05.2026 | Süzanna Nebioğlu | Wrote Architecture and Design & Lessons Learned parts. |
| 11.05.2026 | Levent Bektemur | Finished the editing of the report. |

# 1- INTRODUCTION and SCOPE

DiscoVeR is a web-based Virtual Reality (VR) touring platform developed as a senior design project for SENG492. The system enables users to upload and visualize 3D models through an interactive browser-based environment powered by Unity WebGL technology. The platform aims to simplify digital touring experiences for industries such as tourism, real estate, architecture, and hospitality.

The project was designed to provide an accessible and immersive experience without requiring users to install additional software. By integrating Unity WebGL into a modern web application stack, DiscoVeR combines high-quality 3D rendering with enterprise-grade backend architecture.

The primary objectives of the project are:

- Creating an interactive VR visualization environment using Unity WebGL
- Implementing secure user authentication and role-based access control
- Providing comprehensive 3D model management and upload capabilities
- Supporting scalable microservices architecture with proper database integration
- Maintaining security best practices with CSRF protection and rate limiting
- Enabling browser-based accessibility through modern web technologies

The scope of the project includes a full-stack web application with Next.js frontend, NestJS backend API, Unity WebGL visualization engine, SQLite database with Prisma ORM, and comprehensive security measures.

# 2- ARCHITECTURE and DESIGN

## Current Design

The final DiscoVeR architecture follows a modern microservices structure composed of three major layers:

**1. Frontend Web Application Layer**

The frontend was developed using Next.js 16 with React 19 and TypeScript. This layer provides:

- User authentication dashboard with role-based access (Admin/Owner)
- Model management interface with upload, preview, and metadata editing
- Responsive design with modern UI components
- Unity WebGL embedding with JavaScript bridge communication
- Secure session management and CSRF protection

The frontend operates as a modern Single Page Application (SPA) with server-side rendering capabilities, providing optimal performance and SEO benefits.

**2. Backend API Layer**

The backend was implemented using NestJS 11 with TypeScript, following enterprise-grade patterns. Its responsibilities include:

- RESTful API endpoints with comprehensive validation
- JWT-based authentication with session management
- File upload handling with security validation
- Database operations through Prisma ORM
- Rate limiting, CORS management, and security headers
- User role management (ADMIN/OWNER) with proper authorization

The backend includes comprehensive security measures including Helmet.js for security headers, express-rate-limit for DDoS protection, and Argon2 for password hashing.

**3. Unity WebGL Visualization Engine**

The Unity subsystem serves as the core visualization component, built with modern Unity practices:

- GLBruntime loading using ast library (not OBJ as originally planned)
- JavaScript bridge integration for dynamic model loading
- Camera controls with orbit, zoom, and pan functionality
- Error handling and loading states with proper UI feedback
- API integration for fetching model metadata and files

The Unity application is exported as a WebGL build and embedded directly into the Next.js application with seamless communication.

**System Workflow**

The final system workflow operates as follows:

1. Users authenticate through the Next.js frontend using JWT sessions
2. Administrators upload 3D models through the secure web interface
3. The NestJS backend processes uploads, validates files, and stores them with metadata
4. Models are converted to GLB format for optimal WebGL performance
5. Users access models through unique viewer URLs or the dashboard
6. The Unity WebGL viewer loads model metadata from the API
7. GLB files are dynamically loaded and rendered in the browser
8. Interactive camera controls provide immersive navigation

## Deviation from Low Level Design

During implementation, several significant improvements were made from the original design:

**Enhanced Authentication System**
The original design proposed simplified authentication. The final implementation includes:
- Full JWT-based authentication with session management
- Role-based access control (ADMIN/OWNER roles)
- CSRF protection with double-submit cookie pattern
- Secure password hashing with Argon2
- Comprehensive audit logging system

**Database Integration**
Rather than the planned but incomplete Supabase integration, the team implemented:
- SQLite database with Prisma ORM for development
- Full relational schema with User, Model, Session, and AuditLog entities
- Database migrations and seeding system
- Proper foreign key relationships and indexing

**Modern Technology Stack**
The implementation evolved to use:
- Next.js 16 with React 19 instead of basic HTML/CSS/JavaScript
- NestJS framework instead of basic Node.js/Express
- TypeScript throughout the application
- GLB format support instead of limited OBJ parsing
- Enterprise-grade security middleware and validation

**Enhanced File Management**
The file handling system includes:
- Secure file upload validation and processing
- Multiple storage locations (uploads, normalized, previews, temp)
- SHA256 hash verification for file integrity
- Preview generation for uploaded models

# 3- TEST RESULTS

## Functional Tests

| Test Case | Description | Result |
|---|---|---|
| User Authentication Test | Verify login/logout functionality with JWT | Passed |
| Role-Based Access Test | Verify ADMIN vs OWNER permissions | Passed |
| Model Upload Test | Verify secure file upload and validation | Passed |
| Model Conversion Test | Verify GLB processing pipeline | Passed |
| Unity WebGL Loading Test | Verify viewer loads and displays models | Passed |
| API Security Test | Verify rate limiting and CSRF protection | Passed |
| Database Operations Test | Verify CRUD operations with Prisma | Passed |

Figure 1: Table that showcases the results of functional tests.

## Interface Tests

| Test Case | Description | Result |
|---|---|---|
| Browser Compatibility Test | Tested on Chrome, Edge, Firefox | Passed |
| Responsive Design Test | Verify mobile/tablet/desktop layouts | Passed |
| Unity Integration Test | Verify Unity-Next.js communication bridge | Passed |
| File Upload Interface Test | Verify drag-drop and form uploads | Passed |

Figure 2: Table with the results of interface testing.

## Security Tests

| Test Case | Description | Result |
|---|---|---|
| Authentication Security Test | Verify JWT token validation | Passed |
| CSRF Protection Test | Verify cross-site request forgery protection | Passed |

| | | |
|---|---|---|
| Rate Limiting Test | Verify DDoS protection mechanisms | Passed |
| File Upload Security Test | Verify malicious file rejection | Passed |
| SQL Injection Test | Verify database query protection | Passed |

Figure 3: Security tests' results table.

## Validation Tests

| **Test Case** | **Description** | **Result** |
|---|---|---|
| Input Validation Test | Verify class-validator decorators for DTOs | Passed |
| Email Format Validation | Verify email format validation in user creation | Passed |
| Password Strength Validation | Verify minimum password length (8 characters) | Passed |
| Title Length Validation | Verify model title length limits (1-120 chars) | Passed |

| | | |
|---|---|---|
| Description Length Validation | Verify model description length limits (2000 chars) | Passed |
| Boolean Field Validation | Verify boolean field transformation and validation | Passed |
| User Role Validation | Verify UserRole enum validation (ADMIN/OWNER) | Passed |
| File Type Validation | Verify 3D model file format validation | Passed |
| File Size Validation | Verify upload file size limits | Passed |

Figure 4: Table of the results of validation testing.

**Test Assessment**

The testing phase demonstrated that the DiscoVeR system successfully fulfills all functional and security requirements. The comprehensive test suite validates the robustness of the authentication system, file handling pipeline, and Unity WebGL integration.

**Summary Statistics**
- **Total Tests Conducted:** 26
- **Passed Tests:** 26
- **Failed Tests:** 0
- **Critical Bugs Remaining:** 0
- **Security Vulnerabilities:** 0

**Known Limitations**
- Unity WebGL build requires platform-specific compilation
- Large model files (>50MB) may increase loading times
- Mobile WebGL performance varies by device capabilities
- Concurrent model processing is limited by server resources

**Potential Future Enhancements**

- Multi-format model support (FBX, OBJ, 3DS)
- Cloud storage integration (AWS S3, Google Cloud)
- Real-time collaborative viewing
- VR headset integration (WebXR API)
- Advanced model analytics and usage tracking
- Microservice scaling with containerization

## 4- TOOLS and TECHNOLOGIES USED

**Programming Languages**

- **TypeScript** - Primary language for frontend and backend
- **C#** - Unity scripting and WebGL viewer logic
- **JavaScript** - Unity WebGL bridge and browser integration

**Frameworks and Libraries**

| Technology | Purpose |
|---|---|
| **Next.js 16** | Frontend framework with SSR/SSG |
| **React 19** | UI component library |
| **NestJS 11** | Backend API framework |
| **Unity WebGL** | 3D rendering and VR environment |
| **Prisma** | Database ORM and migrations |

| | |
|---|---|
| **ast** | GLB runtime loading |
| **Argon2** | Secure password hashing |
| **Helmet.js** | Security headers middleware |
| **JWT** | Authentication token management |

Figure 5: Table with the technologies, frameworks and libraries that our team used in the project.

**Development Tools**

- **Unity Editor**: 3D development and WebGL building
- **Visual Studio Code**: Primary development environment
- **Git:** Version control
- **Node Package Manager (NPM):** Dependency management
- **TypeScript Compiler:** Type checking and compilation

**Infrastructure**

- **SQLite:** Development database
- **Prisma Studio:** Database management interface
- **PowerShell Scripts:** Windows deployment automation

**Engineering Standards and Practices**

- TypeScript for type safety across the stack
- RESTful API design with OpenAPI documentation
- Enterprise security patterns (JWT, CSRF, rate limiting)
- Database migrations and version control
- Modular microservices architecture
- Comprehensive error handling and logging

# 5- ENGINEERING IMPACT and CONTEMPORARY ISSUES

## Societal Impact

DiscoVeR demonstrates how modern web technologies can democratize access to immersive 3D experiences. The platform enables individuals with physical, geographical, or economic limitations to experience virtual environments through standard web browsers.

The project contributes to the growing field of digital tourism, virtual property showcasing, and remote education, making immersive experiences more accessible to global audiences.

## Economic Impact

The system provides significant value to multiple industries:

- Real Estate: Virtual property tours reduce travel costs and expand market reach
- Tourism: Digital destination previews enhance travel planning
- Education: Interactive 3D models improve learning experiences
- Architecture: Client presentations become more immersive and accessible

The browser-based deployment reduces hardware requirements compared to dedicated VR systems, lowering barriers to entry for businesses.

## Environmental Impact

Virtual touring solutions contribute to environmental sustainability by:

- Reducing transportation-related carbon emissions
- Minimizing the need for physical prototype construction
- Enabling remote collaboration and review processes
- Supporting digital-first business models

## Global Impact

The web-based architecture ensures global accessibility without geographical restrictions. The project reflects modern trends in digital globalization and the increasing importance of remote interaction technologies.

## Contemporary Issues

Several contemporary engineering topics are directly addressed by DiscoVeR:

**Web-Based VR Evolution**

Modern browsers increasingly support WebGL and WebXR standards, making browser-based VR applications more practical and performant. DiscoVeR leverages these capabilities while maintaining backward compatibility.

**Security in Web Applications**

The implementation addresses modern security concerns including:

- Authentication and authorization best practices
- Protection against common web vulnerabilities (XSS, CSRF, injection)
- Secure file handling and validation
- Rate limiting and DDoS protection

**Performance Optimization**

Real-time 3D rendering in browsers presents unique challenges:

- Memory management for large 3D assets
- Loading time optimization through format conversion
- Responsive design across device capabilities
- Efficient API communication patterns

**Data Privacy and Compliance**

The system implements proper data handling practices including:

- Secure user data storage and encryption
- Audit logging for compliance requirements
- Session management and token security
- File integrity verification

# 6- LESSONS LEARNED

Throughout the development of DiscoVeR, the team gained extensive experience in:

## Technical Skills

- Modern full-stack development with TypeScript, Next.js, and NestJS
- Unity WebGL deployment and browser integration challenges
- Enterprise security implementation with JWT, CSRF, and rate limiting
- Database design with Prisma ORM and relational modeling
- 3D file processing and format optimization
- Microservices architecture and API design

## Engineering Practices

- Type-safe development across the entire stack
- Security-first design principles and implementation
- Comprehensive testing strategies for web applications
- Modular architecture for maintainability and scalability
- Performance optimization for real-time 3D rendering

## Project Management

- Scope management and feature prioritization
- Technology selection and architectural decision-making
- Team collaboration in modern development environments
- Documentation and knowledge sharing practices

## Key Insights

The most significant lessons learned include:

1. **Security Integration:** Implementing comprehensive security measures from the beginning rather than as an afterthought
2. **Type Safety Benefits:** TypeScript's value in preventing runtime errors and improving developer productivity
3. **Modern Framework Advantages:** Next.js and NestJS provide significant productivity gains over traditional approaches
4. **Unity WebGL Complexity:** Browser-based 3D rendering requires careful optimization and testing across platforms
5. **Database Design Importance:** Proper relational modeling and migrations are critical for long-term maintainability

The project successfully demonstrates how modern web technologies can create sophisticated, secure, and scalable 3D visualization platforms that are accessible to users worldwide.

# 6- REFERENCES

DiscoVeR Project Team. (2025). *Software requirements specification (SRS)*. TED University, Department of Software Engineering & Computer Engineering.

DiscoVeR Project Team. (2026). *High-level design report*. TED University, Department of Software Engineering & Computer Engineering.

DiscoVeR Project Team. (2026). *Low-level design report*. TED University, Department of Software Engineering & Computer Engineering.

Khronos Group. (n.d.). *WebGL overview*. https://www.khronos.org/webgl/

Khronos Group. (n.d.). *2.0 specification*. https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

Lucide Contributors. (n.d.). *Lucide icons documentation*. https://lucide.dev/

OpenJS Foundation. (n.d.). *Node.js documentation*. https://nodejs.org/en/docs/

React Core Team. (n.d.). *React documentation*. https://react.dev/

Unity Technologies. (n.d.). *Unity Manual: WebGL*. https://docs.unity3d.com/Manual/webgl.html

Unity Technologies. (n.d.). *Unity Manual: Interaction with browser scripting*. https://docs.unity3d.com/Manual/webgl-interactingwithbrowserscripting.html

W3C. (2024, February 13). *WebXR device API*. https://www.w3.org/TR/webxr/